UD1 - Arquitectura de ordenadores. Estructura funcional

RA1 - Configura equipos microinformáticos, componentes y periféricos, analizando sus características y relación con el conjunto.

Criterios de Evaluación

a) Se han identificado y caracterizado los dispositivos que constituyen los bloques funcionales de un equipo microinformático.

b) Se ha descrito el papel de los elementos físicos y lógicos que intervienen en el proceso de puesta en marcha de un equipo.

c) Se ha analizado la arquitectura general de un equipo y los mecanismos de conexión entre dispositivos.

d) Se han establecido los parámetros de configuración (hardware y software) de un equipo microinformático con las utilidades específicas.

e) Se han evaluado las prestaciones del equipo.

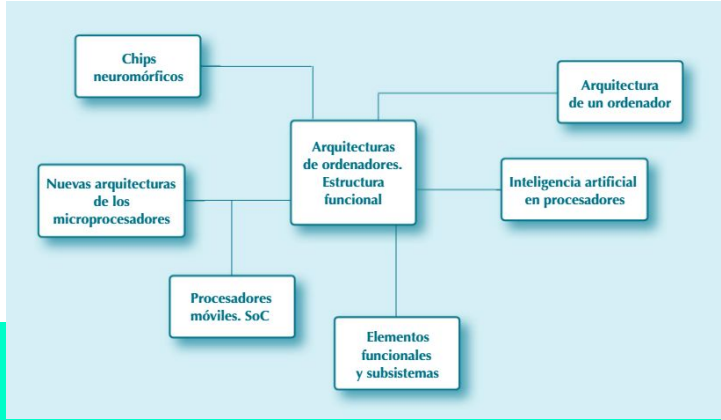
f) Se han ejecutado utilidades de chequeo y diagnóstico.

g) Se han identificado averías y sus causas.

h) Se han clasificado los dispositivos periféricos y sus mecanismos de comunicación.

i) Se han utilizado protocolos estándar de comunicación inalámbrica entre dispositivos.

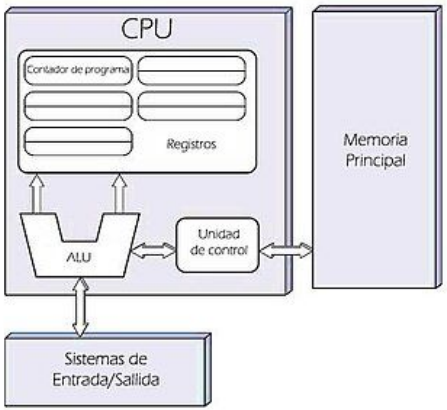
UD1.1 – Arquitectura y subsistemas de un ordenador. Elementos funcionales y subsistemas



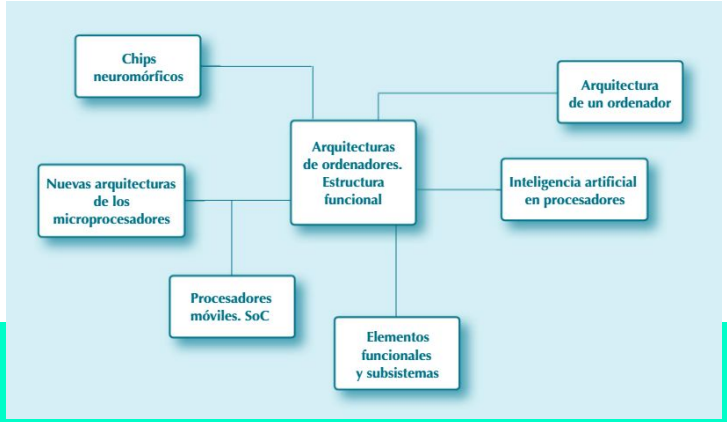
RA1.a, RA1.b, RA1.c y RA1.e.

El modelo de Von Neumann

- Propuesto en 1945 por John Von Neumann.
- Innovación clave: programa y datos se almacenan en memoria.
- Base de todos los ordenadores modernos.
- Estructura en 5 bloques: CPU, memoria, entrada, salida, buses.



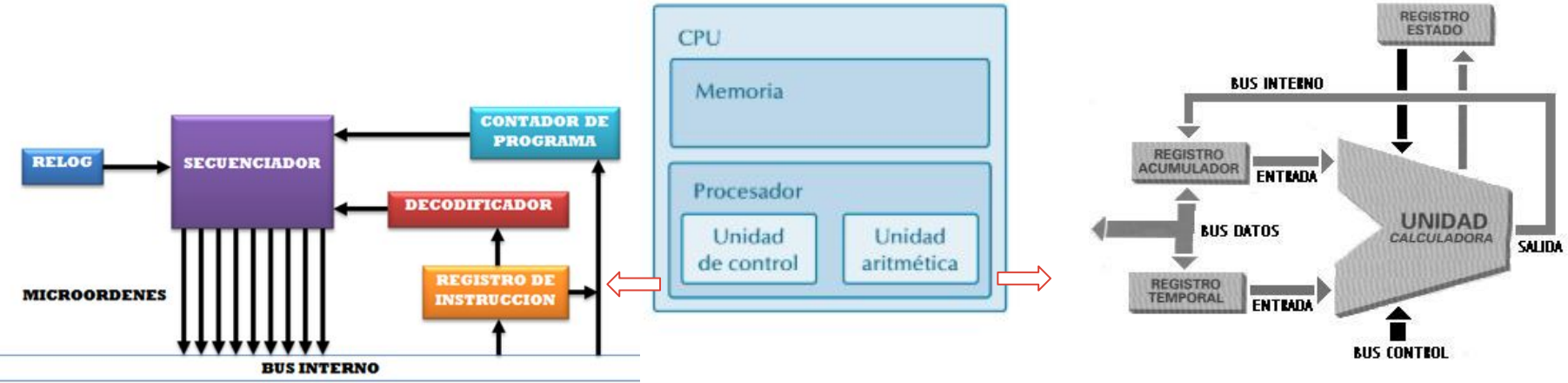
UD1.2 – Arquitectura de un ordenador



RA1.a, RA1.b, RA1.c y RA1.e.

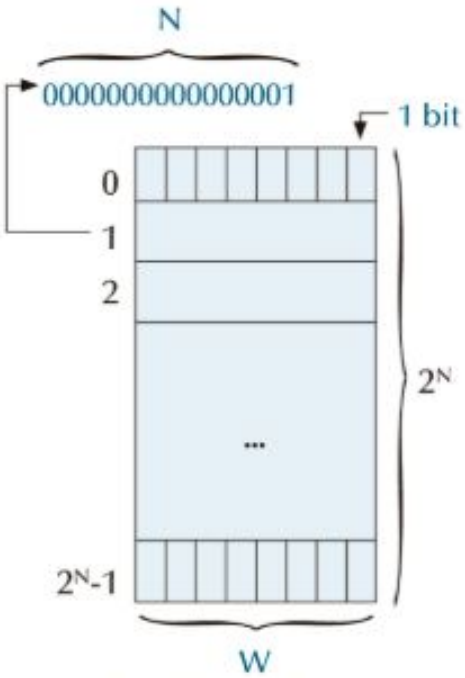
Arquitectura de un ordenador





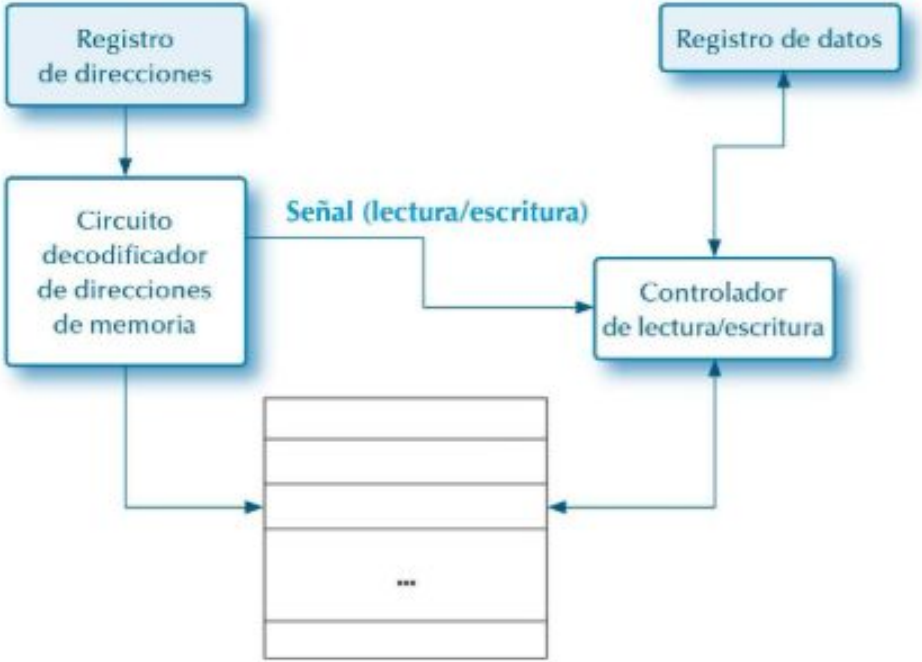
1.2.1. Memoria

La memoria



Direccionamiento

Operaciones de lectura y escritura



Ejercicio: consultar datos memoria RAM y caché

1️⃣ Consultar desde la interfaz gráfica de Windows (GUI)

- 🔹 Capacidad total de memoria RAM instalada
- 🔹 Velocidad de la memoria

2️⃣ Consultar desde la línea de comandos (CLI)

- 🔹 Capacidad total de memoria física instalada (RAM y caché)
- 🔹 Velocidad de la memoria física

En PowerShell podéis sacar info de la CPU parametrizando el CMDlet <Get-CimInstance>:

- **Get-CimInstance -ClassName NombreDeClase | Select-Object Propiedad1, Propiedad2, ...**
- **(Get-CimInstance NombreDeClase).Propiedad**

https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/computer-system-hardware-classes

1.2.2. Subsistema de entrada/salida

- Conjunto de mecanismos que permiten la comunicación entre la CPU/memoria y los periféricos.
- Incluye dispositivos de entrada (teclado, ratón, micrófono), salida (pantalla, impresora) y mixtos (discos, tarjetas de red).
- Gestiona la transferencia de datos mediante controladores, buses y puertos.
- Su función es adaptar la velocidad y formato de los periféricos al sistema, ya que suelen ser mucho más lentos que la CPU y la memoria.

Ejercicio: Identifica los dispositvos de E/S en tu PC

1️⃣ Entrada

Localiza y anota los dispositivos de entrada de tu ordenador

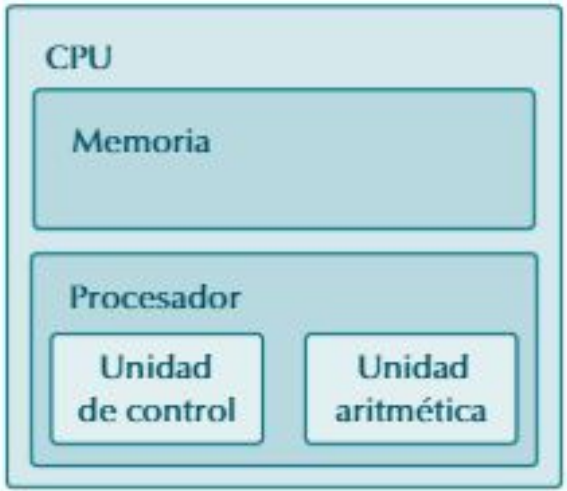
2️⃣ Salida

Identifica los dispositivos de salida

3️⃣ Entrada/Salida (mixtos)

Busca los periféricos mixtos, que sirven tanto para entrada como salida

1.2.3. La CPU: CU y ALU



Está constituida por un único circuito integrado (procesador o micro procesador) e incluye las siguientes unidades:

- Unidad de control (CU)
- Unidad aritmético-lógica (ALU)
- Bloque de registros

Entre las características de la CPU tenemos:

- La frecuencia de reloj
- La longitud de palabra
- La tecnología respecto el juego de instrucciones

- El procesador gestiona el resto de elementos del ordenador comunicándose con ellos a través de los buses (direcciones y datos).
- El uso de microprocesadores está muy extendido en las aplicaciones industriales. Algunos incorporan circuitos específicos para ciertas aplicaciones y llevan integrada una pequeña memoria donde se graba el programa y los datos.

Ejercicio: Consultar datos CPU

1️⃣ Consultar desde la interfaz gráfica de Windows (GUI)

- 🔹 Frecuencia de reloj de la CPU
- 🔹 Longitud de palabra (32 o 64 bits)

2️⃣ Consultar desde la línea de comandos (CLI)

- 🔹 Frecuencia de reloj de la CPU
- 🔹 Longitud de palabra (32 o 64 bits)

En PowerShell podéis sacar info de la CPU parametrizando el CMDlet <Get-CimInstance>:

- **Get-CimInstance -ClassName NombreDeClase | Select-Object Propiedad1, Propiedad2, ...**
- **(Get-CimInstance NombreDeClase).Propiedad**

https://learn.microsoft.com/en-us/windows/win32/cimwin32prov/computer-system-hardware-classes

1.2.4. Registros de la CPU (parte 1)

👉 **Los registros de la CPU son pequeñas memorias ultrarrápidas dentro del procesador que guardan temporalmente datos, direcciones e instrucciones mientras se ejecutan las operaciones.**

Registros visibles al usuario (más bien al programador en lenguaje máquina)

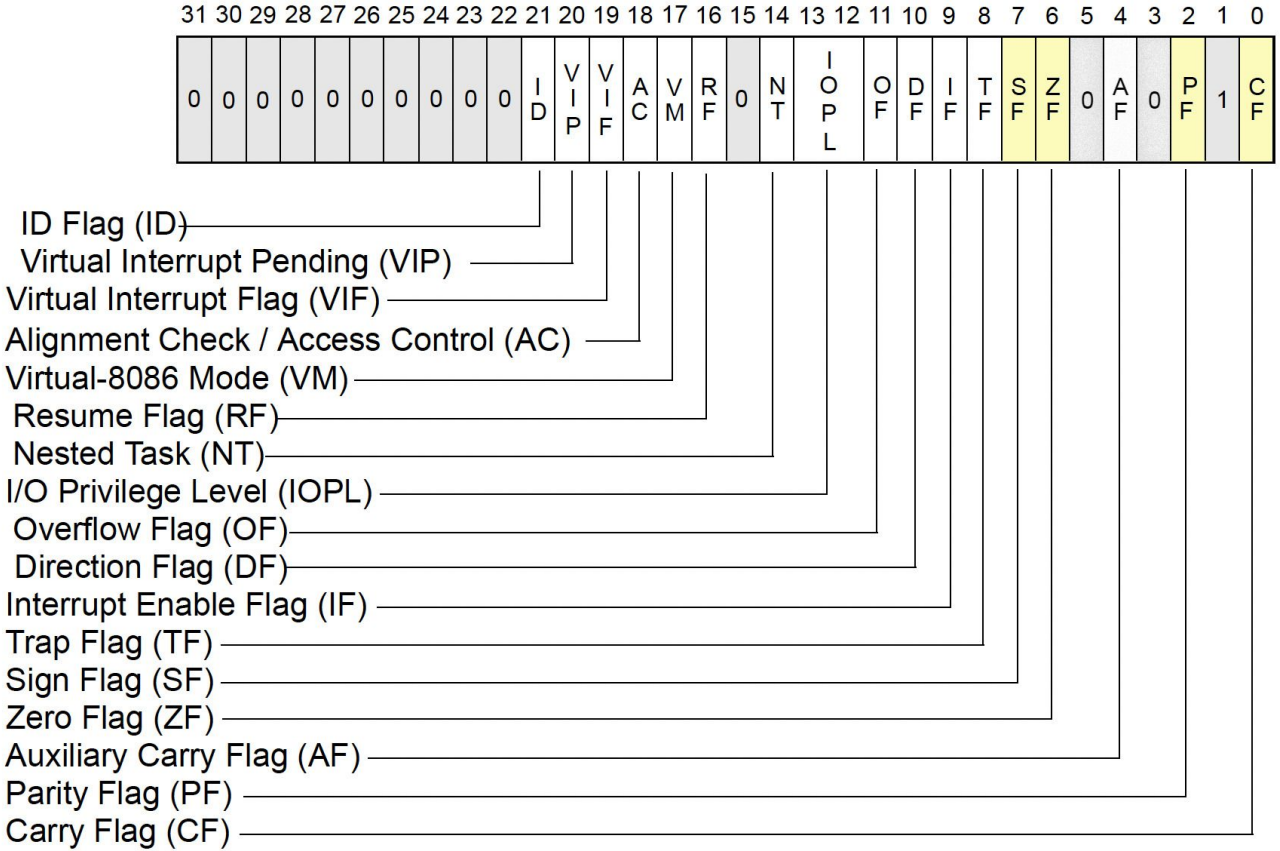
- De dirección: Contienen direcciones de memoria (p.ej. índices y punteros de pila)
- De datos: Contienen datos de operaciones temporales
- De condición: Indican determinadas condiciones (flags) (p.ej: desbordamiento)

Tipo de registro	Nombre(s) comunes	Uso principal
De datos	RAX (acumulador), RBX, RCX, RDX	Operaciones aritméticas, lógicas, resultados temporales. RAX suele actuar como acumulador.
De direcciones / punteros	RSI (source index), RDI (destination index), RBP (base pointer), RSP (stack pointer), RIP (instruction pointer)	Apuntan a posiciones de memoria, pilas, direcciones de código o datos.
De propósito general	R8 – R15	Uso flexible para datos o direcciones, según la instrucción.
De condición (flags)	RFLAGS	Almacena el resultado de la última operación (cero, acarreo, signo, desbordamiento, etc.). Muy importante para saltos condicionales.

Registros de la CPU (parte 2)

Registro	Nombre	Función principal
RSP	*Stack Pointer* (puntero de pila)	Apunta siempre a la cima de la **pila**. La pila es una zona de memoria donde se guardan direcciones de retorno, variables temporales, etc. Cada vez que haces un `PUSH` o `CALL`, RSP baja; con `POP` o `RET`, RSP sube.
RBP	*Base Pointer* (puntero base)	Marca una referencia fija dentro del marco de pila de una función. Sirve para acceder a variables locales y parámetros de manera ordenada. Muy usado por compiladores en depuración.
RSI	*Source Index* (índice fuente)	Usado en instrucciones de cadenas (string instructions) para apuntar al origen de datos en memoria. Ejemplo: `MOVSB` copia un byte desde [RSI] a [RDI]. RSI se auto-incrementa/decrementa.
RDI	*Destination Index* (índice destino)	También usado en instrucciones de cadenas, pero apunta a la **dirección destino** en memoria. Ejemplo: en `MOVSB`, RDI recibe el byte que viene de [RSI].
RIP	*Instruction Pointer* (puntero de instrucción)	Guarda la dirección de la **siguiente instrucción** que la CPU debe ejecutar. Es el “program counter”. Al hacer un salto, llamada o interrupción, se actualiza RIP.

Registros de la CPU (parte 3)



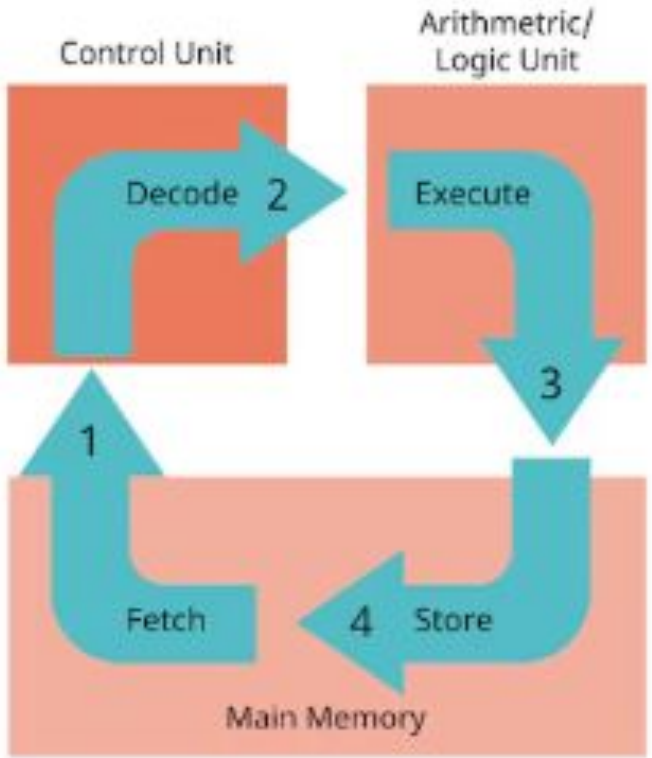
Registros de la CPU (parte 4)

Registro	Nombre completo	Función principal
PC / IP / RIP	*Program Counter* / *Instruction Pointer*	Guarda la dirección de la **siguiente instrucción** que debe ejecutarse.
IR / RI	*Instruction Register*	Contiene la **instrucción actual** que está siendo decodificada/ejecutada.
MAR	*Memory Address Register* (Registro de Dirección de Memoria)	Almacena la **dirección de memoria** a la que se quiere acceder (lectura o escritura).
MDR / MBR	*Memory Data Register* / *Memory Buffer Register*	Contiene el **dato que entra o sale de memoria** en la operación de lectura/escritura.
RFLAGS / PSW	*Flags Register* o *Program Status Word*	Indica el estado de la última operación (cero, acarreo, signo, desbordamiento, interrupciones, etc.).
CS	*Control Register(s)*	Registros especiales para gestionar modos de operación, protección de memoria, paginación, etc. (ej. CR0, CR3 en x86).

1.2.5. El ciclo de instrucción

Ciclo de instrucción (simplificado)

1. **Fetch (búsqueda):**
    - El **Contador de Programa (CP/PC/RIP)** indica la dirección de la siguiente instrucción.
    - Esa dirección se pasa al **MAR** (Registro de Dirección de Memoria).
    - La memoria devuelve la instrucción a través del **MDR** (Registro de Datos de Memoria).
    - La instrucción se guarda en el **RI** (Registro de Instrucción).
2. **Decode (decodificación):**
    - El **RI** se interpreta → la **Unidad de Control** genera las señales necesarias.
3. **Execute (ejecución):**
    - Se buscan operandos (de memoria o registros).
    - La **ALU** realiza la operación.
    - El resultado se guarda (en registros o memoria).
    - El **CP** se actualiza para apuntar a la siguiente instrucción.



Ejercicio: Simulando el ciclo de instrucción

Imagina que la CPU de un ordenador debe ejecutar la instrucción:

`SUMAR A = B + C`

1. Fase de búsqueda (Fetch): Indica qué registros y componentes intervienen para que la instrucción llegue desde memoria hasta la CPU.
2. Fase de decodificación (Decode): Explica qué hace la UC con esa instrucción.
3. Fase de ejecución (Execute): Señala cómo la ALU utiliza los registros para realizar la operación.
4. Fase de escritura (Write-back): Explica dónde queda almacenado el resultado final.

Trabajo en grupos pequeños:
Cada grupo representa con un esquema o breve guion qué hace cada bloque (UC, ALU, registros, memoria) en cada fase.

1.2.6. Arquitectura CISC vs arquitectura RISC

ARQUITECTURA CISC (pocas instrucciones complejas)	ARQUITECTURA RISC (muchas instrucciones pequeñas)
- Formatos de instrucción de varios tamaños	- Formatos de instrucción de pocos tamaños
- Interpreta microinstrucciones	- Interpreta microoperaciones
- Muchos modos de direccionamiento	- Pocos modos de direccionamiento
- Pocos registros de propósito general	- Muchos registros de propósito general
- Repertorio de instrucciones flexible	- Repertorio de instrucciones rígido.
- Son lentas. Ejecución por software	- Son rápidas. Ejecución directa por Hardware
- UC microprogramada.	- UC cableada.

Aplicaciones actuales:

Hoy en día, **CISC** (Intel, AMD) se usa sobre todo en PCs y servidores, mientras que **RISC** (ARM, RISC-V) domina en móviles, tablets e IoT por su eficiencia energética.

En la práctica ya no hay una división tan estricta:

Los procesadores CISC modernos traducen instrucciones complejas a micro-operaciones simples (estilo RISC).

Los RISC modernos han incorporado instrucciones más complejas.

Resultado: la mayoría de CPUs actuales son un híbrido que mezcla lo mejor de ambos mundos.

1.3. SoC: procesadores para dispositivos móviles

SoC = System on a Chip

- **Definición:** Circuito integrado que reúne en un único chip los principales componentes de un ordenador o dispositivo.

- **Componentes típicos:**
  - CPU (procesador)
  - GPU (procesador gráfico)
  - Controladores de memoria y E/S
  - Módulos de comunicación (WiFi, Bluetooth, 5G)
  - Aceleradores específicos (IA, seguridad, multimedia)

- **Ventajas:**
  - Menor consumo energético
  - Ocupa menos espacio → ideal para móviles y dispositivos embebidos
  - Integración de todo el sistema en un único chip

- **Ejemplos:**
  - Smartphones (Qualcomm Snapdragon, Apple M1/M2, Samsung Exynos)
  - Consolas portátiles, IoT, tablets